Motivation

Abstract Data Type Dictionary is a collection of key-value pairs where the keys are required to be unique. Some of the methods that a dictionary must implement include

`Dict.Init()`	Initializes the dictionary
`Dict.Insert(`**Key** K, **Data** d`)`	Inserts (K, d) in dictionary.
Data `Dict.Retrieve(`**Key** K`)`	Return the data associated with Key K.
bool `Dict.Member(`**Key** K`)`	Return **true** if Key K is in the dictionary.

Introduction and Definitions

Definition 1. A **hash table** is an array H of size m,

$H[0]$	$H[1]$	$H[2]$	$\cdots$	$H[m-2]$	$H[m-1]$

with a hash function h : Key $K \to \{0, 1, 2, \ldots, m-2, m-1\}$ that maps keys to locations in the array H.

```
function HashTable.Insert(Key K, Data D)
    i = h(K)
    Insert (K, D) in H[i]
end function
```

Definition 2. A hash table implements **modular (division) hashing** if the hash function has the form

$$h(K) = K \mod m \tag{1}$$

Problems:

- If $m = 1000$, then $(23, 1023, 2023, 3023, \ldots, 9023)$ all map to $H[23]$.
- If m is even and all keys K are even, then $h(K)$ is always even ($H[i]$ is never accessed for any value of i that is odd.
- If m is a multiple of 10 and all keys K are multiples of 10, then $h(K)$ is always a multiple of 10 (i.e. only $1/10^{th}$ of the table is ever access)

Rule of thumb: m should always be prime!

Definition 3. A hash table implements **multiplicative hashing** if the hash function has the form

$$h(K) = \lfloor m \times \text{frac}(\sigma K) \rfloor \tag{2}$$

where $\sigma \in [0.5, 1]$ and frac(x) returns the fractional part of x (i.e. frac(1.8234) = 0.8234)

Problems:

- If $h(K) = \lfloor m \times \text{frac}((7/8) \times K) \rfloor$, then beware of multiples of 8.
- If $h(K) = \lfloor m \times \text{frac}((12/17) \times K) \rfloor$, then beware of multiples of 17.

Idea: σ should approximate an irrational number!
A common choice:

$$h(K) = \left\lfloor m \times \text{frac}\left(\frac{\sqrt{5}-1}{2} \times K\right)\right\rfloor$$
$$h(K) \approx \left\lfloor m \times \text{frac}\left(\frac{2654435769}{2^{32}} \times K\right)\right\rfloor$$

Definition 4. A hash table implements **universal hashing** if the hash function has the form

$$h(K) = ((aK + b) \mod p) \mod m \tag{3}$$

for some large prime $p \gg m$

- **Theorem:** If a, b are randomly chosen integers from $[1, p-1]$, then $\Pr(h(K_1) = h(K_2)) = 1/m$ for any pair of keys $K_1, K_2 < p$

Question: What makes a good hash function and what makes a bad hash function? Some bad functions include

- $h(K) = 5K \mod 1000$
- $h(K) = 6K \mod 333$

These functions are bad because gcd$(5, 1000) = 5$ and gcd$(6, 333) = 3$. The first hash function will only use $1/5$ of the available locations and the second only $1/3$ of the available locations. Good hash functions should have the following properties

- All hash table locations are equally likely to be accessed.
- Keys with a "regular" pattern should not be mapped to the same locations.

A Hash Table is simply an array

	0	1	2	3	4	5	6
H							

with a corresponding hash function,

$$h(K) = 3K \mod 7.$$

Suppose we execute `HashTable.Insert`$(1, D_1)$. First, we must compute the hash of the key,

$$h(1) = 3 \mod 7 = 3$$

and then insert the element at location $h(1) = 3$,

	0	1	2	3	4	5	6
H				$(1, D_1)$			

Next, suppose we perform

- `HashTable.Insert`$(4, D_4)$ which inserts at location $h(4) = 12 \mod 7 = 5$
- `HashTable.Insert`$(12, D_{12})$ which inserts at location $h(4) = 36 \mod 7 = 1$
- `HashTable.Insert`$(10, D_{10})$ which inserts at location $h(4) = 30 \mod 7 = 2$

and the Hash Table becomes

	0	1	2	3	4	5	6
H		$(12, D_{12})$	$(10, D_{10})$	$(1, D_1)$		$(4, D_4)$	

Next, suppose we perform `HashTable`$(3, D_3)$. Then $h(3) = 9 \mod 7 = 2$, but location 2 is already occupied by $(10, D_{10})$[1]. We introduce a definition to describe this event,

Definition 5. A **Hash Table collision** occurs when $h(K_1) = h(K_2)$, but $K_1 \neq K_2$. Hash Table implementations must implement some method of resolving these collisions.

[1] We note that this was almost certainly going to happen since the domain of the hash function is presumably much larger than $[0, m-1]$, therefore the pigeonhole principle guarantees that such an occurrence was inevitable.

Chained hashing

Chained Hashing resolves collisions by modifying so that every $H[i]$ is a linked list and when a collision occurs the colliding element is appended to the linked list at location $H[i]$.

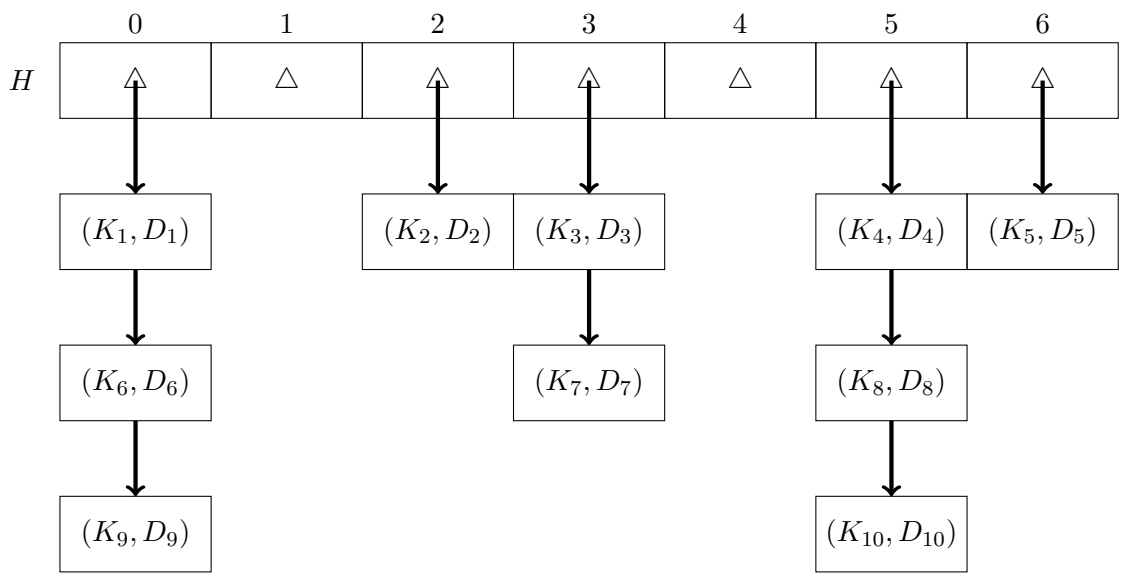


Figure 1: An example of a Hash Table of size 7 implemented using Chained Hashing

Notice that each element of H now only contains a `head` pointer that points to the first element of the linked list at the location. Furthermore, notice that this Hash Table now contains more elements than there are entries in the table – this is an advantage that Chained Hashing has over some other collision resolution methods. We can modify the rudimentary `HashTable.Insert` method introduced earlier to accommodate our changes,

```
function HashTable.Insert(Key K, Data D)
    i = h(K)
    Append (K, D) to linked list H[i]
end function
```

Best Case	$\Theta(1)$
Expected Case	$\Theta(1)$
Worst Case	$\Theta(1)$

Analyzing the runtime of this method is very easy since we assume that the hash function takes constant time to evaluate and appending an element to a linked list also takes constant time, therefore the running time is in $\Theta(1)$ for the best, worst, and expected case.

Example 1. Suppose the Hash Table in Figure 1 uses the hash function

$$h(K) = 3K \mod 7.$$

Perform the following operation

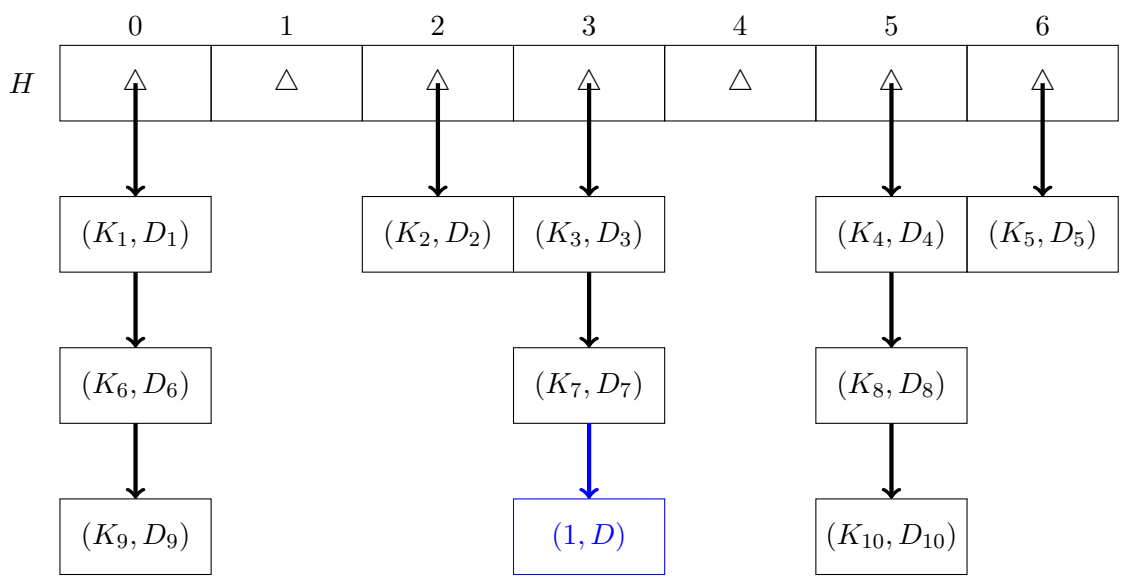
- `HashTable.Insert`$(1, D)$

on the Hash Table and draw the result.

We begin by computing the location of the insertion,

$$h(1) = 3 \mod 7 = 3$$

and then must append $(1, D)$ to the linked list at that location,



Some other standard methods for Dictionaries that we mentioned at the start of the notes are `Member` and `Retrieve`. Both of these methods function by finding a particular key (or at least attempting to find the key) within a Hash Table. In particular,

- `HashTable.Member`(Key K) returns true if key K is in the Hash Table.
- `HashTable.Retrieve`(Key K) returns the data associated with key K in the Hash Table.

Consider the following implementations,

```
function HashTable.Member(Key K)
    i = h(K)
    if K is in linked list H[i] then
        return true
    else
        return false
    end if
end function
```

```
function HashTable.Retrieve(Key K)
    i = h(K)
    Retrieve (K, D) from linked list H[i]
    return D
end function
```

Notice how we have failed to specify how to search the linked lists – in general we can only search through the linked lists in linear order, which will affect the running time of these methods. The running time of these methods is proportional to the length of the linked list at $H[i]$, so if we can determine the best, worst, and expected length of these linked lists, then we can determine the running time! We begin by defining *two* notions of "size",

- Let m be the size of the Hash Table.
- Let n be the number of elements currently in the Hash Table.

Best case The best case running time occurs when the length of the linked list is 0 for `Member` and 1 for `Retrieve`. In both cases the best case running time is in $\Theta(1)$.

Worst case The worst case running time occurs when every element is in a single linked list **and** the desired key is at the tail of the linked list. In this case the algorithm must search every location in the linked list and takes $\Theta(n)$ time.

Expected case To determine the expected time, we must determine the expected length of the linked list at location $H[i]$. In order to do this, we will introduce an "indicator variable' to indicate whether key K_j is in location i. Let

$$X_{ij} = \begin{cases} 1 & \text{if } h(K_j) = i \\ 0 & \text{otherwise} \end{cases}$$

This random variable indicates whether K_j is in location i. To determine the number of elements at location i we need only sum these indictors for over each of the keys in the Hash Table. Define

$$X_i = \sum_{j=1}^{n} X_{ij}$$

as the length of the linked list at location i. Therefore, $E[X_i]$ is the expected length of the linked list at location i. To compute this expected value we must first assume that our hash function is "well-behaved" enough such that all locations are equally likely. Expressed mathematically, we assume that **Prob** $(h(K_j) = i) = \frac{1}{m}$. So by linearity,

$$E[X_i] = E\left[\sum_{j=1}^{n} X_{ij}\right] = \sum_{j=1}^{n} E[X_{ij}]$$

and by definition

$$E[X_{ij}] = \textbf{Prob}\,(h(K_j) = i) \cdot 1 + \textbf{Prob}\,(h(K_j) \neq i) \cdot 0 = \frac{1}{m}$$

Therefore,

$$E[X_i] = E\left[\sum_{j=1}^{n} X_{ij}\right] = \sum_{j=1}^{n} E[X_{ij}] = \sum_{j=1}^{n} \frac{1}{m} = \frac{n}{m}$$

and the expected running time is

$$ET(m,n) = c + cE(X_i) = c + c \cdot \frac{n}{m}.$$

Thus,

$$\boxed{ET(m,n) \in \Theta\left(1 + \frac{n}{m}\right).}$$

```
function HashTable.Member(Key K)
    i = h(K)
    if K is in linked list H[i] then
        return true
    else
        return false
    end if
end function
```

Best Case	$\Theta(1)$
Expected Case	$\Theta(1 + \frac{n}{m})$
Worst Case	$\Theta(n)$

```
function HashTable.Retrieve(Key K)
    i = h(K)
    Retrieve (K, D) from linked list H[i]
    return D
end function
```

Best Case	$\Theta(1)$
Expected Case	$\Theta(1 + \frac{n}{m})$
Worst Case	$\Theta(n)$

Some other similar methods can also be implemented. `Add` adds the value x to the data associated with key K and `Replace` replaces the data associated with key K with the new data D,

```
function HashTable.Add(Key K, Data x)
    i = h(K)
    Find element e in linked list H[i]
    e.data = e.data + x
end function
```

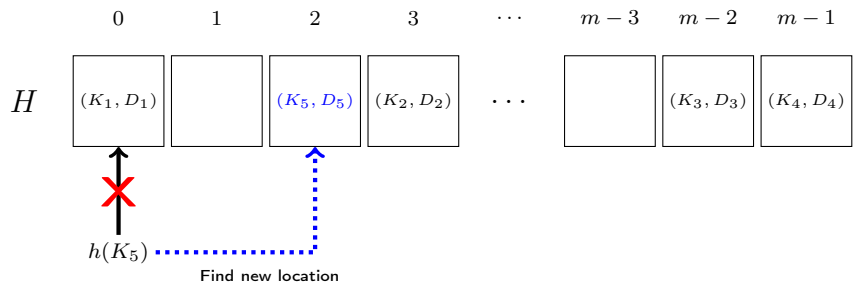
Best Case	$\Theta(1)$
Expected Case	$\Theta(1 + \frac{n}{m})$
Worst Case	$\Theta(n)$

```
function HashTable.Replace(Key K, Data D)
    i = h(K)
    Find element e in linked list H[i]
    e.data = D
end function
```

Best Case	$\Theta(1)$
Expected Case	$\Theta(1 + \frac{n}{m})$
Worst Case	$\Theta(n)$

Open address hashing

Open Address Hashing resolves collisions by attempting to find a new location once a collision occurs.



To do this, we modify our hash function to have two arguments to indicate the number of "rehashes" attempted. For example,

$$h(K, j) = (K + j) \mod m$$

where the second argument j corresponds to the number of rehashes. j is initially 0 and is incremented whenever a collision occurs while attempting to insert key K. Three implementations of this idea are

Linear probing $h(K, j) = (h_1(K) + j) \mod m$

Quadratic probing $h(K, j) = (h_1(K) + c_1 j + c_2 j^2) \mod m$

Double hashing $h(K, j) = (h_1(K) + j \cdot h_2(K)) \mod m$

We can implement `HashTable.Insert` exactly as described above,

```
function HashTable.Insert(Key K, Data D)
    j = 0
    repeat
        i = h(K, j)
        if H[i] is empty then
            H[i] = (K, D)
            return
        else
            j = j + 1
        end if
    until (j = m)
    error: "hash table overflow"
end function
```

Example 2. Given the Hash Table

	0	1	2	3	4	5	6
H		$(12, D_{12})$	$(10, D_{10})$	$(1, D_1)$		$(4, D_4)$	

and corresponding hash function

$$h(K, j) = (3K + 2j) \mod 7$$

Perform the following operation

- `HashTable.Insert`$(5, D_5)$

on the Hash Table and draw the result.

We begin computing the location with $j = 0$,

$$h(5, 0) = (15 + 0) \mod 7 = 1$$

and we notice that location 1 is already occupied by $(12, D_{12})$. That means we increment j by 1 and recompute the hash function,

$$h(5, 1) = (15 + 2) \mod 7 = 3.$$

Again we notice that location 3 is already occupied by $(1, D_1)$ so we must again increment j by 1 and recompute the hash function,

$$h(5, 2) = (15 + 4) \mod 7 = 5.$$

which is *again* already occupied, this time by $(4, D_4)$. So we increment j by 1 and recompute the hash function,

$$h(5, 3) = (15 + 6) \mod 7 = 0$$

and see that location 0 is unoccupied! So, we insert $(5, D_5)$ at location $h(5, 3) = 0$,

	0	1	2	3	4	5	6
H	$(5, D_5)$	$(12, D_{12})$	$(10, D_{10})$	$(1, D_1)$		$(4, D_4)$	
	$h(5, 3)$	$h(5, 0)$		$h(5, 1)$		$h(5, 2)$	

Unlike Chained Hashing, insertion is no different than our other standard methods in terms of running time. In order to determine if a key is present or to find the data corresponding to a key, we must perform an *identical* number of hashes. That is, if it took l hashes to insert an element, it will also take l hashes to retrieve that element. In addition to this, the implementation of all three standard methods is extremely similar!

```
function HashTable.Member(Key K)
    j = 0
    repeat
        i = h(K, j)
        if H[i].key = K then
            return true
        end if
        j = j + 1
    until (j = m) or (H[i] is empty)
    return false
end function
```

```
function HashTable.Retrieve(Key K)
    j = 0
    repeat
        i = h(K, j)
        if H[i].key = K then
            return H[i].data
        end if
        j = j + 1
    until (j = m) or (H[i] is empty)
    return null
end function
```

The question now becomes: what is the expected number of rehashes required? We again define $m =$ size of the Hash Table and $n =$ number of elements in the Hash Table.

Best case The best case running time occurs when only a single hash is required, in which case *all* of the methods take $\Theta(1)$ time.

Worst case The worst case occurs when there is a collision with every element present in the Hash Table, in which case *all* of the methods take $\Theta(n)$ time.

Expected case For the sake of convenience we proceed as if we are only analyzing the `HashTable.Insert` method, but the analysis is identical for the other methods. We begin by defining

$$X = \text{number of times } H[i] \text{ is not empty on line 5}$$

and (as with Chained Hashing) we assume that our hash function is "well-behaved" enough such that all locations are equally likely. Let

$$i_j = h(K, j)$$

be the jth location probed. Under our "well-behavedness" assumptions,

$$\textbf{Prob}\,(H[i_0] \text{ is not empty}) = \frac{n}{m}$$

since there are m locations, n of which are already occupied! Furthermore if each rehash is guaranteed to probe a new location, then there are $n-1$ possible elements to collide with in a total of $m-1$ positions, so

$$\textbf{Prob}\,(H[i_0] \text{ and } H[i_1] \text{ are not empty}) = \frac{n}{m} \cdot \frac{n-1}{m-1} \leq \left(\frac{n}{m}\right)^2$$

and

$$\textbf{Prob}\,(H[i_0], H[i_1], \text{ and } H[i_2] \text{ are not empty}) = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \frac{n-2}{m-2} \leq \left(\frac{n}{m}\right)^3.$$

In general,

$$\textbf{Prob}\,(H[i_0], H[i_1], \ldots, H[i_{q-1}] \text{ are not empty}) = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \cdots \cdot \frac{n-q+1}{m-q+1} \leq \left(\frac{n}{m}\right)^q.$$

Thus,

$$\textbf{Prob}\,(X \geq q) = \textbf{Prob}\,(H[i_0], H[i_1], \ldots, H[i_{q-1}] \text{ are not empty}) \leq \left(\frac{n}{m}\right)^q$$

And using our theorem for non-negative probabilities,

$$E[X] = \sum_{q=1}^{m} \textbf{Prob}\,(Y \geq q) \leq \sum_{q=1}^{m} \left(\frac{n}{m}\right)^2 \leq \sum_{q=1}^{\infty} \left(\frac{n}{m}\right)^2 = \frac{\frac{n}{m}}{1 - \frac{n}{m}}.$$

We cannot further simplify this expression with making assumptions on the "load factor" of the Hash Table. The **load factor** α of a hash table is $\frac{n}{m}$.

Suppose that the Hash Table is "less than half full"; that is, assume that

$$n \leq \frac{m}{2} \qquad \Longrightarrow \qquad \frac{n}{m} \leq \frac{1}{2}.$$

Under this assumption we have that

$$E[X] \leq \frac{\frac{1}{2}}{1 - \frac{1}{2}} = 1$$

That is, the expected number of rehashes is 1, which is quite good! Later in this course we will discuss how you can implement a Hash Table to guarantee that the load factor remains bounded $\frac{1}{2}$ regardless of the number of insertions performed, but for now that remains a mystery. The expected running can be expressed as

$$ET(m, n) = c + cE[X] \leq c + c = 2c$$

Therefore,

$$\boxed{ET(m, n) \in \Theta(1).}$$

Deleting an element from a Hash Table implemented with Chained Hashing is relatively straightforward – simply remove the element from the linked list, no harm, no foul. We are not so fortunate when deleting an element from a Hash Table implemented using Open Address Hashing, because we may cause elements to appear to be missing that are actually present. For example, given the Hash Table

	0	1	2	3	4	5	6
H	$(5, D_5)$	$(12, D_{12})$	$(10, D_{10})$	$(1, D_1)$		$(4, D_4)$	

with hash function $h(K, j) = (3k + 2j) \mod 7$, suppose we deleted $(1, D_1)$ from the Hash Table

	0	1	2	3	4	5	6
H	$(5, D_5)$	$(12, D_{12})$	$(10, D_{10})$			$(4, D_4)$	

and then execute `HashTable.Retrieve`$(5, D_5)$. We compute $h(5, 0) = 1$, but $H[1]$.`key` $\neq 5$, so we continue. We next compute $h(5, 1) = 3$ and $H[3]$ is empty. We would therefore have to conclude that 5 is not a key in the Hash Table. Oh no! To resolve this issue, we introduce a "flag" to signal whether a location was **ever** occupied. This guarantees that if we find an empty location that was once occupied we know that we may require further searching to find our desired key. We must therefore modify all of our methods to include this flag.

```
function HashTable.Insert(Key K, Data D)
    j = 0
    repeat
        i = h(K, j)
        if H[i] is empty or (H[i].flagDeleted = true) then
            H[i] = (K, D)
            H[i].flagDeleted = false
            return
        else
            j = j + 1
        end if
    until (j = m)
    error: "hash table overflow"
end function
```

Best Case	$\Theta(1)$
Expected Case	$\Theta(1)$
Worst Case	$\Theta(n)$

```
function HashTable.Retrieve(Key K)
    j = 0
    repeat
        i = h(K, j)
        if (H[i].key = K) and (H[i].flagDeleted = false) then
            return H[i].data
        end if
        j = j + 1
    until (j = m) or (H[i] is empty)
    return null
end function
```

Best Case	$\Theta(1)$
Expected Case	$\Theta(1)$
Worst Case	$\Theta(n)$

```
function HashTable.Member(Key K)
    j = 0
    repeat
        i = h(K, j)
        if H[i].key = K and (H[i].flagDeleted = false) then
            return true
        end if
        j = j + 1
    until (j = m) or (H[i] is empty)
    return false
end function
```

Best Case	$\Theta(1)$
Expected Case	$\Theta(1)$
Worst Case	$\Theta(n)$

```
function HashTable.Delete(Key K)
    j = 0
    repeat
        i = h(K, j)
        if H[i].key = K then
            H[i].flagDeleted = true
            return true
        end if
        j = j + 1
    until (j = m) or (H[i] is empty)
    return false
end function
```

Best Case	$\Theta(1)$
Expected Case	$\Theta(1)$
Worst Case	$\Theta(n)$

Applications

```
function ContainsDuplicates(A[],n)
    for i = 2 to n do
        for j = 1 to i − 1 do
            if A[i] = A[j] then
                return true
            end if
        end for
    end for
    return false
end function
```

Let $T(n)$ be the running time of `ContainsDuplicates` on inpute of size n. The best case running time is $\Theta(1)$ when $A[1] = A[2]$, but the worst case running time on an array with no duplicates is $\Theta(n^2)$. We will ignore the expected case for now, but the question becomes: can we do any better using hashing? Consider the implementation given below where we use hashing to "keep track" of the values found thus far. If we find a new element we insert it into a hash table and if we find a duplicate we return true.

```
function ContainsDuplicates(A[],n)
    HashTable.Init()
    for i = 1 to n do
        if HashTable.Member(A[i]) then
            return true
        else
            HashTable.Insert(A[i],true)
        end if
    end for
    return false
end function
```

Let $T(n)$ be the running time of `ContainsDuplicates` on input of size n. The best case running time is $\Theta(1)$ again when $A[1] = A[2]$. To analyze both the expected running time and the worst case running time we assume that A has no duplicates. Furthermore, let $T_{in}(s)$ be the running time of `HashTable.Insert`(K, D) in a hash table with s elements and let $T_{mem}(s)$ be the running time of

`HashTable.Member`(K) in a hash table with s elements. We can therefore express the running time as

$$T(n) = \sum_{i=1}^{n} (T_{in}(s) + T_{mem}(s))$$

Finally, we assume that the hash table is implemented using Open Address Hashing and that $m \geq 2n$

Worst case The worst case running time of $T_{in}(s)$ and $T_{mem}(s)$ is cs since the running time is proportional to the number of elements in the hash table. Furthermore, in the worst case a new element is inserted each iteration and therefore $s = i$. So,

$$T(n) = \sum_{i=1}^{n} (T_{in}(s) + T_{mem}(s)) = \sum_{i=1}^{n} (ci + ci) = 2c \sum_{i=1}^{n} i = 2c \frac{n(n+1)}{2}$$

Thus, the worst case running time remains $\Theta(n^2)$.

Expected case Since $m \geq 2n$, we know that $T_{in}(s)$ and $T_{mem}(s)$ both take $\Theta(1)$ time. Therefore by linearity,

$$ET(n) = E\left(\sum_{i=1}^{n} (T_{in}(s) + T_{mem}(s))\right) = \sum_{i=1}^{n} (ET_{in}(s) + ET_{mem}(s)) = \sum_{i=1}^{n} (c + c) = 2cn$$

So the expected case running time is $\Theta(n)$ even when assuming the array has no duplicates!